

Prompt Templates

Copy-paste prompts for each pipeline stage. Replace `{placeholders}` with runtime values. Works with any LLM API (OpenAI, Anthropic, Gemini, local models).

Stage 1: Knowledge Extractor

Per-chunk summary

```
You are analyzing part {chunk_index} of {total_chunks} of a software repository.
```

```
Repository structure:  
{repo_structure}
```

```
Source contents (this chunk):  
{chunk_content}
```

```
Summarize this chunk. Include:  
- Key functionalities and components  
- Important files and their roles  
- Architectural insights  
- Notable design patterns
```

```
Keep the response modular so it can be merged with other chunks.  
Do not generate diagrams. Output prose only.
```

Final merge

```
Merge these partial repository summaries into one cohesive architectural report.
```

```
Partial summaries:  
{partial_summaries}
```

```
Requirements:  
- Remove repetition  
- Logical flow from overview to components to integrations  
- Cohesive picture of functionality and architecture  
- Maximum 4000 tokens; prioritize the most critical architectural information
```

```
Do not generate diagrams. Output prose only.
```

Stage 2: Metadata Assembler

Use when the user request is informal and metadata must be inferred.

Given this architectural summary and user request, produce a complete view specification as JSON.

Summary:
{summary}

User request:
{user_request}

Output a JSON object with all fields from the view specification schema:
summary, Concern, Behavior, Granularity, Architecture Scope,
Architectural Notation,
Components Nature, Connectors Nature, QAs, Architectural Styles,
Shapes, Colored?, Connectors Direction, Legend?, Nested Components?,
Explicit Ports/Interfaces?, Explicit Connectors?

Rules:

- Infer missing fields with sensible defaults
- Prefer medium granularity unless user specifies otherwise
- Match notation to user preference; default to boxes_and_arrows
- For dynamic behavior, set Behavior to "dynamic"
- List any assumptions in a separate "assumptions" array

Output valid JSON only.

Stage 3: Prompt Builder

Produces the generation prompt from view specification. This is the core ArchView pattern.

You are an expert software architect. Generate valid Python code that produces an architecture diagram.

Architectural knowledge
{summary}

View specification

- Concern: {Concern} – the diagram must visually emphasize this
- Behavior: {Behavior}
- Granularity: {Granularity} – respect this abstraction level; do NOT over-detail
- Components: {Components Nature}
- Connectors: {Connectors Nature}
- Quality attributes: {QAs}
- Scope: {Architecture Scope}

- Notation: {Architectural Notation}
- Styles: {Architectural Styles}
- Visual: shapes={Shapes}, colored={Colored?}, connector direction={Connectors Direction}
- Layout: legend={Legend?}, nested={Nested Components?}, ports={Explicit Ports/Interfaces?}, explicit connectors={Explicit Connectors?}

Output requirements

1. Use the {library} library as the ONLY representation mechanism
 2. Output ONLY executable code – no markdown fences, no explanations
 3. The summary is the primary guide for structure and relationships
 4. Show architectural abstractions, not individual classes or files
 5. Include all primary components and relationships for the stated scope
 6. If scope is "part", show only the relevant subsystem
 7. Clear component names; readable left-to-right or top-to-bottom layout {if diagrams}
 8. For diagrams library: use ONLY imports from this whitelist:
{import_whitelist}
 9. Refer to <https://diagrams.mingrammer.com/docs/nodes/> for valid node classes
- {endif}

Generate the diagram code now.

Library selection:

Architectural Notation	{library} value
icons_and_arrows	diagrams
boxes_and_arrows	graphviz
UML	plantuml
boxes	graphviz

Stage 4: View Generator (PlantUML variant)

Use when notation is UML and you want PlantUML text directly (no Python wrapper).

Static (component diagram)

You are an expert software architect. Generate a PlantUML component diagram.

Repository summary:
{summary}

Concern: {Concern}
Granularity: {Granularity}

Scope: {Architecture Scope}
Styles: {Architectural Styles}

Requirements:

- Show components and their relationships
- Emphasize the specified concern
- Respect granularity – no class-level explosion
- Valid PlantUML only: @startuml ... @enduml
- No explanations, only code

Dynamic (sequence diagram)

You are an expert software architect. Generate a PlantUML sequence diagram.

Repository summary:
{summary}

Concern: {Concern}
Behavior: dynamic

Requirements:

- Show runtime message flow between components
- Label messages clearly
- Match the described system behavior
- Valid PlantUML only: @startuml ... @enduml
- No explanations, only code

Stage 5: Error correction (compile-verify retry)

Inject when rendering fails.

A previous diagram generation attempt failed. Fix the code.

Error:
{error_message}

Previous code:
{previous_code}

Original view specification:
{view_spec_json}

Requirements:

- Fix the specific error
- Keep the same architectural intent

- Output ONLY corrected executable code
- No markdown fences, no explanations

Stage 6: Quality Checker

Use as a separate critic agent or self-reflection pass.

You are evaluating an architecture diagram against its specification.

View specification:

{view_spec_json}

Diagram (code or description):

{diagram_output}

Evaluate each criterion with rating (Meets / Partially Meets / Does Not Meet) and one-sentence justification:

1. Clarity – readable labels, logical layout, descriptive names
2. Consistency – uniform notation and connector styles
3. Completeness – primary components and relationships present for scope
4. Accuracy – relationships match summary and concern
5. Level of Detail – matches granularity, not too vague or too detailed

Also check for known failure modes:

- Over-detailing (too many low-level elements)
- Too many components for stated granularity
- Missing primary components from summary
- Extra components outside scope

Output JSON:

```
{
  "clarity": {"rating": "", "justification": ""},
  "consistency": {"rating": "", "justification": ""},
  "completeness": {"rating": "", "justification": ""},
  "accuracy": {"rating": "", "justification": ""},
  "level_of_detail": {"rating": "", "justification": ""},
  "overall_pass": true/false,
  "revision_needed": "specific instructions if overall_pass is false"
}
```

Stage 7: User-facing response

Template for final agent output (any framework).

Architecture View

[Rendered diagram or diagram-as-code]

View Specification

Field	Value
Concern	{Concern}
Behavior	{Behavior}
Granularity	{Granularity}
Notation	{Architectural Notation}
Scope	{Architecture Scope}

Assumptions

{assumptions_list}

Limitations

{limitations_list}
